Final Year Project

App for designing communication switches report

Student Bhavya Bhatia

Student ID: 21206612

A thesis submitted in part fulfilment of the degree of

BSc. (Hons.) in Computer Science

Supervisor: Professor John Murphy



UCD School of Computer Science

University College Dublin

April 24, 2026

Table of Contents

1 **Project Specification** . . . 3

2 **Introduction** . . . 8

3 **Related Work and Ideas** . . . 10

3.1 Literature Review . . . 10

4 **Data Considerations** . . . 14

4.1 Nature and Scale of Data . . . 14

4.2 Source and Characteristics of Data . . . 14

4.3 Rationale for Data Usage . . . 15

4.4 Data Validation and Processing . . . 15

4.5 Licensing, Privacy, and Ethical Considerations . . . 15

5 **System Design and Algorithmic Approach** . . . 16

5.1 Design Choices . . . 16

5.2 Network Topology Algorithms and Calculation . . . 17

6 **Project Workplan** . . . 25

6.1 Project Timeline and Phases . . . 25

6.2 Summary of the Workplan . . . 27

7 **Summary and Conclusions** . . . 28

7.1 Future Work . . . 28

7.2 Testing and Feedback . . . 29

Abstract

This project aims to develop an iOS application for network topology design, specifically focusing on Clos and Slepian switching networks. The app will serve as a user-friendly tool enabling users to input network parameters, such as size, and receive optimized configurations and visual representations of the topology. Targeted at network engineers, researchers, and students, the application seeks to simplify the complex process of designing and visualizing efficient switching networks. Built using SwiftUI and leveraging intuitive UI/UX design principles, the app will feature robust algorithms to calculate optimal configurations and interactive visualizations to aid understanding. By combining technical rigor with accessibility, this project aims to bridge the gap between theoretical network design and practical implementation, empowering users with an efficient, easy-to-use tool.

Repository of the project (GitLab link): GitLab

Chapter 1: Project Specification

The main demands of the project, as outlined in the original specification, are as follows:

Core Goals:

- User Input Handling: Enable users to input network size, stages, and other topology-related parameters as shown in Figure 1.4.

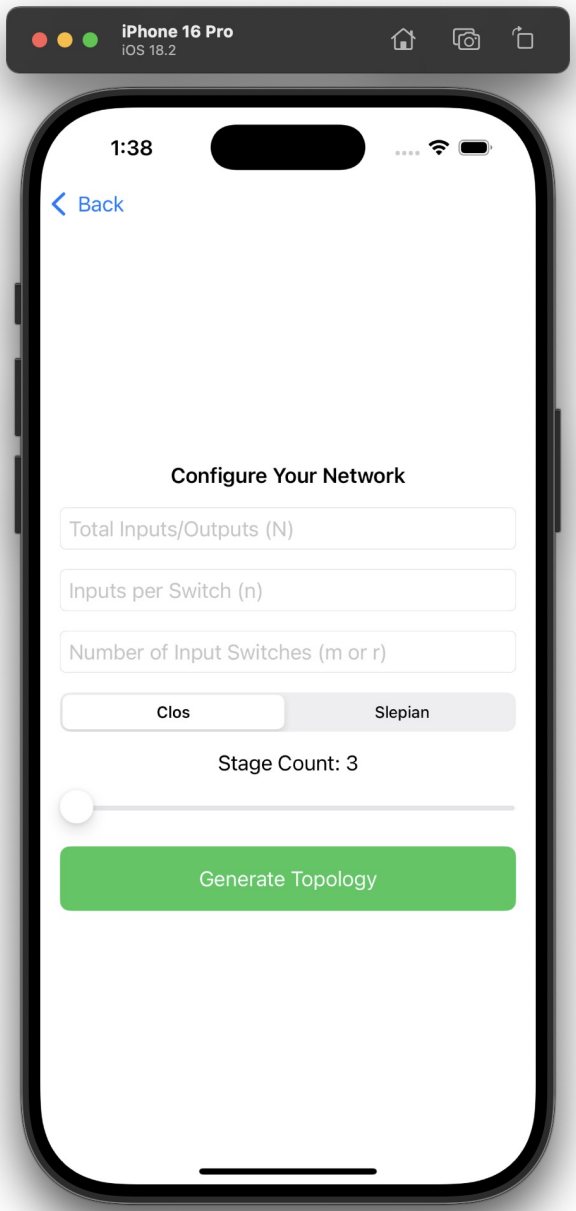


Figure 1.1: Screenshot of the Network Input Form

- Algorithm Implementation: Develop algorithms for creating Clos and Slepian network topologies.
- Visualization: Integrate real-time, interactive visualizations of the topologies using SwiftUI's Canvas API or Core Graphics.

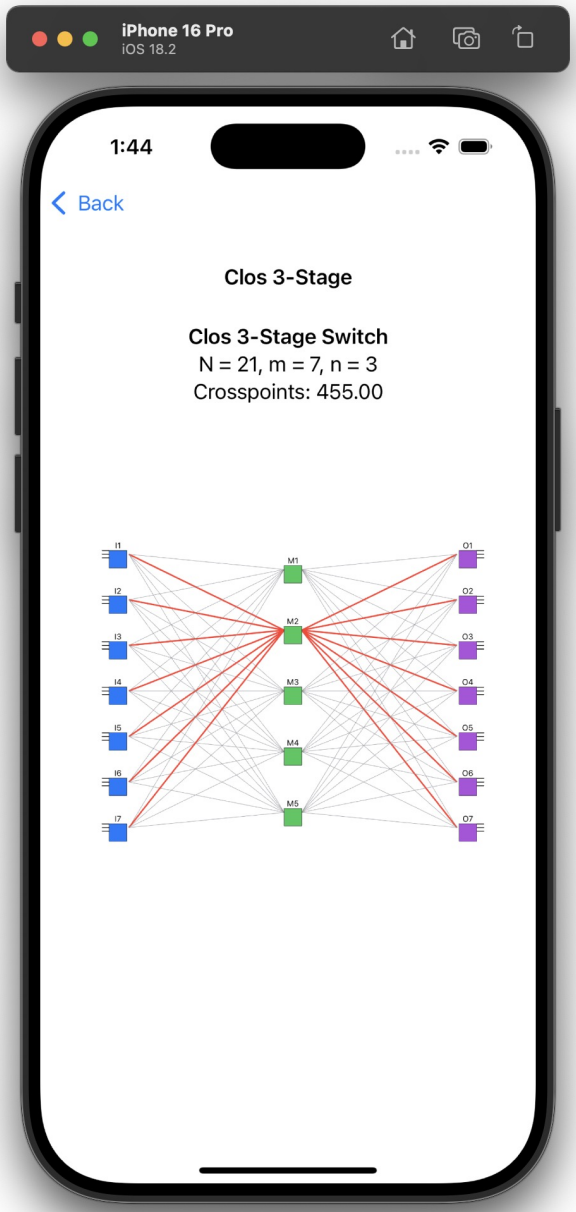


Figure 1.2: Screenshot of a basic 3 Stage Clos Network

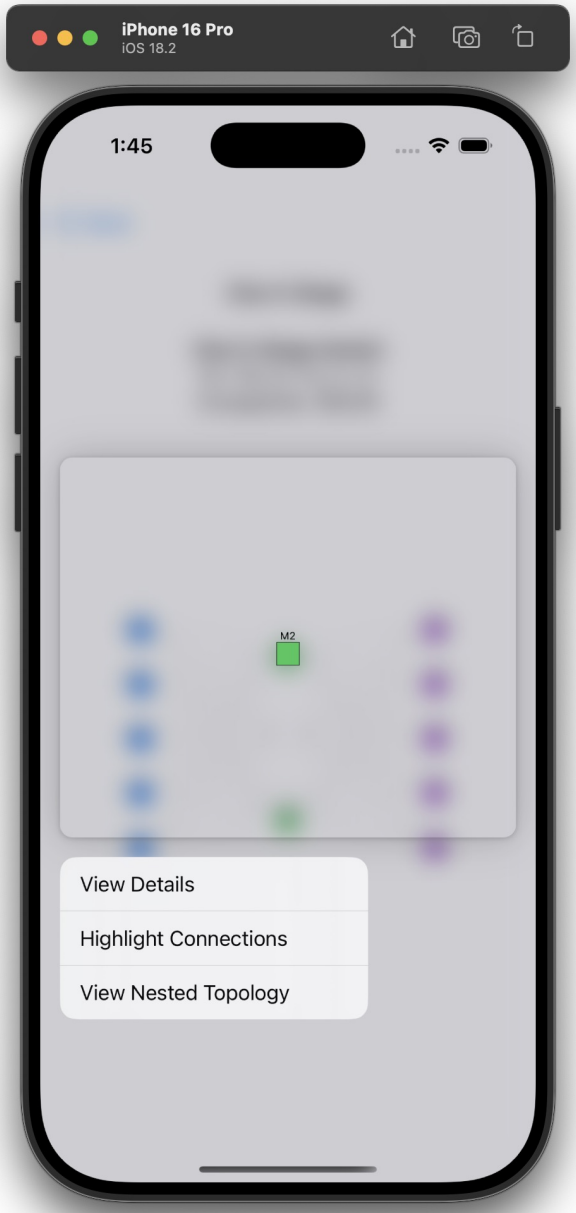


Figure 1.3: Screenshot of the ability to press and hold on a switch to view details

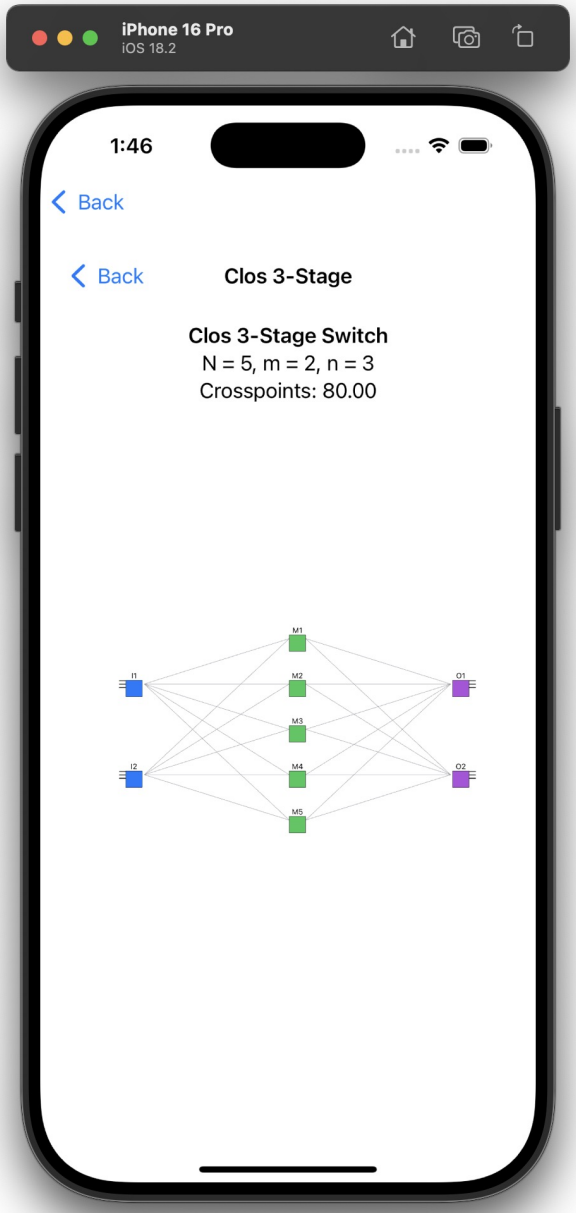


Figure 1.4: Screenshot of how a nested topology is shown

- Error Handling: Ensure robust error handling and app performance for varying network sizes.
- Modular Code: Implement modular logic for calculations, including switch size and stage number.

Advanced Goals:

- Performance Optimization: Ensure smooth performance and scalability for larger networks.
- A/B Testing: Conduct usability and feature comparison testing to refine the user experience.
- Advanced Features: Add optional features such as export options for network topologies and integrated help guides.

- Final Testing and Documentation: Perform comprehensive debugging, user testing, and prepare detailed documentation, including user instructions and a final project report.

Chapter 2: **Introduction**

In today's increasingly connected world, computer networks serve as critical infrastructure underpinning countless technological advancements and services, including telecommunications, cloud computing, distributed systems, and data center operations [1, 2]. Efficient and optimized network designs are paramount for sustaining the high-speed data traffic and scalability required by modern computing applications. Among various switching network architectures used today, two have notably stood out due to their robustness, efficiency, and scalability: the Clos and Slepian (also known as Beneš) switching networks [3].

Originally formalized by Charles Clos in 1953, the Clos network provides a highly efficient solution for non-blocking communications in large-scale switching fabrics, proving critical in telecommunications and large data center environments due to its ability to scale and reliably route data with minimal latency and resource usage [1]. Similarly, the Slepian network, first proposed by David Slepian [4] and later refined by Václav Beneš [3], offers a rearrangeable non-blocking switching architecture. Its inherent flexibility makes it particularly suitable for dynamically changing connection requirements, as commonly encountered in multi-processor systems and parallel computing environments.

Despite their proven effectiveness, the process of designing and optimizing these switching networks remains complex and challenging. Current network-design solutions often fall into two distinct categories: either overly technical tools designed for expert users, demanding substantial theoretical and practical knowledge, or overly simplistic diagrammatic tools lacking the necessary computational rigor required for practical implementation. This leaves a significant gap for tools that are simultaneously powerful, accurate, and accessible to both experts and non-experts alike [2].

This project seeks to bridge this gap through the development of an intuitive iOS application designed specifically for the simplified creation, optimization, and visualization of Clos and Slepian switching networks. Leveraging the latest advancements in software development and visualization frameworks, particularly SwiftUI [5], this application will offer an interactive platform for users to specify network parameters, including network size, number of inputs and outputs, and connectivity requirements. From these inputs, the application will calculate and generate optimal network configurations, determining essential parameters such as the ideal number of stages, the optimal arrangement of switches, and the minimum number of required crosspoints to ensure non-blocking operation.

Furthermore, a distinctive aspect of this project is the integration of real-time graphical representations, presenting users with clear, interactive visualizations of the resulting network topologies using SwiftUI's powerful rendering capabilities, such as the Canvas view [6]. This interactive element not only facilitates immediate comprehension of the constructed networks but also significantly simplifies the task of network optimization, offering an intuitive means of understanding the complex interactions and parameters involved.

The overarching objective of this project extends beyond merely automating the technical aspects of Clos and Slepian network designs. By providing clear visual representations and accessible interactions, the proposed application aims to serve an educational function as well, enabling network engineers, students, and researchers to deepen their understanding of fundamental network design principles. Consequently, the project is expected to democratize access to advanced network configuration techniques, potentially empowering a broader range of users and contributing to innovation and advancements in both academic research and practical network implementations.

Given the continual evolution of networking technology, particularly in telecommunications and data centers, this project aligns closely with contemporary trends and needs, addressing both technical precision and usability. By simplifying the network design and optimization process without sacrificing computational rigor, this application stands to significantly improve workflow efficiency, reduce configuration time, and foster broader engagement with advanced network topologies.

Ultimately, the vision of this project is to create an accessible yet powerful tool that transforms the design of Clos and Slepian switching networks from a specialized task into an intuitive, interactive, and visually engaging process, thus supporting innovation in the broader field of telecommunications and networking.

Chapter 3: Related Work and Ideas

3.1 Literature Review

3.1.1 Domain Overview and Foundation

The development of a network topology design application, focusing on Clos and Slepian switching networks, necessitates a robust understanding of switching principles and methodologies. The lecture materials provided by the professor offer foundational insights into the concepts of network switching, topology optimization, and the practical considerations for implementing efficient and scalable networks. Clos and Slepian switches, recognized for their pivotal roles in telecommunications and data centers, serve as the primary focus due to their ability to minimize crosspoints while ensuring optimal connectivity. The materials emphasize the hierarchical design of switches, performance evaluation metrics, and recursive optimization strategies, forming the theoretical backbone of this project [2].

3.1.2 Existing Solutions and Applications

The field of network design and visualization is supported by a variety of tools and applications, ranging from comprehensive network simulators and emulators to more general-purpose diagramming software. These tools cater to different needs within the networking domain, from planning and testing complex enterprise networks to creating simple visual representations for documentation or educational purposes.

Network simulation and emulation tools like Cisco Packet Tracer and GNS3 are widely used for designing, configuring, and testing network behavior in a virtual environment [7, 8]. These applications offer detailed simulations of various networking protocols and devices, allowing users to build and troubleshoot complex network scenarios. They are particularly valuable in educational settings and for network professionals to model and validate configurations before deployment.

In parallel, diagramming tools such as Microsoft Visio and Draw.io (diagrams.net) provide a flexible platform for creating visual representations of networks using pre-defined shapes and connectors [9]. While not offering simulation capabilities, these tools are useful for documenting existing network infrastructure, planning logical topologies, and communicating network designs to others. They often feature drag-and-drop interfaces and libraries of network-specific icons.

3.1.3 Analysis of Solutions

While existing network design tools provide valuable functionalities, they often present limitations that can hinder their accessibility and specific applicability to the detailed design and visualization of specialized network architectures like Clos and Slepian switches.

Simulation tools, despite their power, typically come with a steep learning curve and can be resource-intensive, requiring significant computational power and technical expertise to set up and

operate effectively [8]. They are often desktop-based, limiting their portability and accessibility. Furthermore, while some may allow for the representation of Clos or Slepian-like structures, they may not have built-in functionalities for calculating optimal parameters or visualizing the unique recursive nature of these networks from a theoretical design perspective.

General-purpose diagramming tools, on the other hand, are often more user-friendly and accessible, but they lack the computational intelligence required for network design optimization [9]. Users must manually input all network parameters and draw the topology, which can be a tedious and error-prone process, especially for complex or recursive structures. These tools serve primarily as drawing boards rather than intelligent design aids. Additionally, many professional-grade diagramming tools require paid licenses, which can be a barrier for students and individual researchers. Some online tools exist, but they may require a stable internet connection, which can limit their usability in certain environments.

Both categories of tools often lack a dedicated focus on the theoretical underpinnings and specific calculations required for designing optimal Clos and Slepian networks, necessitating manual computation of parameters before visualization.

3.1.4 Justification for a Mobile Application

The limitations of existing desktop-bound simulation tools and computationally-lacking diagramming software highlight a need for a more accessible, focused, and intelligent tool specifically designed for Clos and Slepian network topology design and visualization. A mobile application addresses several of these shortcomings and offers unique advantages.

Firstly, a mobile application provides unparalleled portability and accessibility, allowing users to design and visualize networks on the go, without being tied to a desktop environment or requiring high-end computing resources. The pervasive nature of smartphones and tablets makes a mobile app a convenient platform for students, researchers, and professionals alike.

Secondly, a dedicated mobile app can offer a streamlined and intuitive user interface specifically tailored for inputting network parameters and interacting with topology visualizations. Leveraging the touch-based interface of mobile devices can simplify the design process compared to traditional desktop applications.

Thirdly, by focusing specifically on Clos and Slepian networks, the application can embed the necessary algorithms and optimization logic directly within the app, providing users with calculated optimal configurations and detailed information alongside the visual representation. This eliminates the need for manual calculations or relying on separate tools. The application can guide users through the design process and provide instant feedback on the network's properties.

Finally, a well-designed mobile application can potentially offer offline functionality for core design and visualization tasks, making it usable in environments where internet connectivity is limited or unavailable. This contrasts with many online diagramming tools that require a persistent connection.

Therefore, developing an iOS application for designing optimal switching networks fills a valuable niche by offering a portable, accessible, computationally-intelligent, and user-friendly tool specifically tailored to the unique requirements of Clos and Slepian topologies, addressing limitations found in existing, more general-purpose solutions.

3.1.5 Tools and Development Frameworks

The choice of tools and frameworks is critical in transforming theoretical knowledge into a functional application. After evaluating various options, SwiftUI [5] was selected as the primary framework for developing this iOS application. SwiftUI is Apple's modern UI toolkit, designed to build user interfaces declaratively and efficiently. Compared to UIKit and traditional Swift frameworks, SwiftUI offers several advantages that align with the project's objectives:

- Modern and Declarative Syntax: SwiftUI's declarative approach simplifies the development of interactive and dynamic interfaces, crucial for the visualization of network topologies.
- Integration with Swift: Although SwiftUI is built on Swift, it eliminates the need for verbose boilerplate code, enabling developers to focus on core functionalities.
- Efficiency and Performance: SwiftUI is optimized for performance on Apple devices, ensuring smooth rendering of real-time visualizations of complex network configurations.
- Future-Proofing: As Apple's recommended framework for iOS development, SwiftUI ensures long-term support and compatibility with upcoming OS updates.
- Ease of Learning: Leveraging SwiftUI reduced the learning curve for this project. The framework is well-documented and supported by a large community, providing ample resources for resolving development challenges.
- By adopting SwiftUI, the project benefits from enhanced productivity, improved user interface capabilities, and alignment with modern iOS development practices.

3.1.6 Why SwiftUI Over UIKit and Swift

While UIKit has been a robust framework for iOS app development for over a decade, SwiftUI supersedes it in several critical aspects. UIKit's imperative programming model often results in verbose and complex code, especially when managing dynamic and interactive content. Conversely, SwiftUI's declarative nature allows for a more streamlined and readable codebase, which is particularly advantageous when implementing the complex visualizations required for this project. Additionally, UIKit lacks the seamless integration with real-time previews and the ability to adapt efficiently to device-specific constraints, both of which are key strengths of SwiftUI.

Furthermore, while traditional Swift is a powerful language for backend logic, the integration of SwiftUI reduces dependency on additional libraries and tools for UI development. This combination of functionality and simplicity makes SwiftUI the ideal choice for this project, enabling a focus on building the core algorithms for switch optimization without being bogged down by intricate UI coding.

3.1.7 Minimal Additional Reading

The lecture materials [2] provided sufficient domain knowledge for understanding the workings of Clos and Slepian switches. Topics such as the hierarchical reasoning in switch design, optimization of crosspoints, and recursive network configurations were comprehensively covered. This foundational understanding reduced the need for extensive external research. Additionally, Swift documentation served as a reliable resource for implementing the app, offering detailed explanations and examples on how to utilize SwiftUI's features effectively. The clarity and depth of these materials significantly streamlined the development process, allowing the focus to remain on practical implementation rather than theoretical exploration.

3.1.8 Summary and Value

In summary, this project draws on a well-established foundation of switching concepts and leverages state-of-the-art tools like SwiftUI to achieve its goals. The choice of SwiftUI not only aligns with the project's technical requirements but also reflects a commitment to using efficient, modern, and sustainable development practices. Supported by thorough lecture materials and official Swift documentation, the project is poised to deliver a robust application that simplifies network topology design and visualization, meeting both the technical and usability demands of its target audience.

Chapter 4: Data Considerations

In the context of this project, 'data' primarily refers to the input provided by the user and the subsequent computational outputs generated by the application's algorithms. No external datasets are collected, stored, or analyzed.

4.1 Nature and Scale of Data

The data consists of numerical inputs provided by the user, specifically:

- Total Inputs/Outputs (N)
- Inputs per Switch (n)
- Number of Input Switches (m or r)
- Number of Stages (k)
- Switch Type (Clos or Slepian)

Based on these inputs, the application computes and generates output data including:

- Optimal or specified values for n, m, and r.
- The calculated total number of crosspoints for the network.
- The structural parameters of the network topology (number of switches per stage, switch dimensions, interconnections).

The scale of the input data is relatively small, consisting of a few integer values and a string. The scale of the generated output data depends on the network size (N) and the number of stages (k), which determines the number of switches and connections to be calculated and visualized. While the number of individual data points for calculations is limited, the complexity and amount of data required for visualization can increase with larger networks. The application is designed to handle network sizes up to a predefined maximum (e.g., $N = 10,000$ as mentioned in your outline of approach).

4.2 Source and Characteristics of Data

The primary source of data is direct user input via the application's interface. This is unstructured user data in the sense that it is entered by the user, but the application's design anticipates and requires specific types of input (numerical for network parameters). The data is static during

a single instance of network design and visualization; it does not change dynamically unless the user modifies the input. The data is processed locally within the application on the user's device and is not streamed or stored remotely. The output data (calculated parameters and topology) is structured according to the defined models for Clos and Slepian switches. The mathematical formulas underpinning these calculations, which operate on the input data to produce the output data, are discussed in detail in a later section (see Section 5.2).

4.3 Rationale for Data Usage

The user inputs are fundamental to defining the specific network topology the user wishes to design and visualize. The application's purpose is to take these user-defined parameters and apply established mathematical formulae and algorithms to compute the characteristics of the corresponding Clos or Slepian network. The generated output data is essential for providing the user with the optimized configuration details and for rendering the accurate visual representation of the network topology. No other external data would be relevant or necessary for this specific functionality, as the project focuses on the theoretical design and visualization based on user-specified constraints.

4.4 Data Validation and Processing

Input validation is crucial to ensure the application functions correctly and to prevent errors. The application implements validation to check that inputs are of the expected type (integers for numerical parameters) and fall within reasonable ranges (e.g., number of stages must be appropriate for the chosen switch type, and network size must be a positive integer). Error handling is in place to inform the user of invalid inputs. The core processing involves applying the algorithms detailed in the "Outline of Approach" chapter to the validated user inputs to calculate the network parameters and topology structure.

4.5 Licensing, Privacy, and Ethical Considerations

Since the project does not involve collecting, using, or distributing any external or personal data, there are no licensing restrictions, privacy concerns, or ethical issues related to data collection or usage. The data processed by the application is solely the input provided by the user for the purpose of network design and the results generated by the application's calculations. This data is not stored persistently by the application or shared with any external service.

Chapter 5: System Design and Algorithmic Approach

This chapter details the design choices, algorithms, and implementation approach adopted for the iOS application. The focus is on developing a user-friendly tool that accurately calculates and visualizes the topology of Clos and Slepian switching networks based on user-defined parameters.

5.1 Design Choices

5.1.1 Framework

SwiftUI was selected for its modern, declarative syntax, which allows for dynamic UI design and seamless integration with Apple's ecosystem [5]. SwiftUI ensures future compatibility and provides robust tools for real-time graphical rendering of network topologies.

5.1.2 Target Platform

The app will be built for iOS, focusing on devices like iPhones and iPads to maximize accessibility for users in educational and professional settings.

5.1.3 User Interface (UI)

- Input Form: A clean, intuitive interface allows users to specify network parameters such as total network size (N), inputs per switch (n), number of input switches (m or r), and the number of stages (k). The application provides flexibility by allowing users to leave one of N, n, or m/r blank for automatic calculation based on the others, or to specify all three (with validation to ensure consistency, as implemented in `InputView.swift`).
- Visualization: Graphical representations of Clos and Slepian networks are generated dynamically based on the calculated or user-defined parameters. This utilizes SwiftUI's Canvas API for real-time rendering [6]. The visualization aims to clearly depict the stages, switches within each stage, and the interconnections between them.
- Help Section: A built-in guide assists users unfamiliar with switch configurations or topology optimization concepts (as implemented in `HelpView.swift`).

5.2 Network Topology Algorithms and Calculation

The core of the application lies in the accurate implementation of algorithms to calculate the parameters and topology of Clos and Slepian networks. This section outlines the mathematical foundations and the approach taken in the application's Swift code (`Calculator.swift`).

5.2.1 Clos Network Design

The Clos network is a multi-stage switching network designed to be non-blocking, meaning that a connection can always be established between any idle input and any idle output regardless of the existing connections within the network [1]. A standard Clos network consists of three stages: an input stage, a middle stage, and an output stage. For larger networks, this structure can be recursively applied, where the middle stage is itself a smaller Clos network.

A single-stage crossbar switch with N inputs and N outputs requires N^2 crosspoints. This can be visualized as a grid where each input line intersects with each output line, with a switch at each intersection point (Figure 5.1).

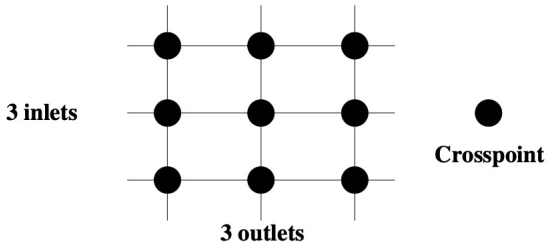


Figure 5.1: Single-Stage Crossbar Switch with 3^2 Crosspoints

The topology of a 3-stage Clos network is defined by three parameters: N, the total number of inputs/outputs; n, the number of inputs per switch in the input stage (and outputs per switch in the output stage); and m, the number of switches in the input and output stages. For a strictly non-blocking network, the number of switches in the middle stage is $2n - 1$ [1]. A general representation of a 3-stage Clos network is shown in Figure 5.2.

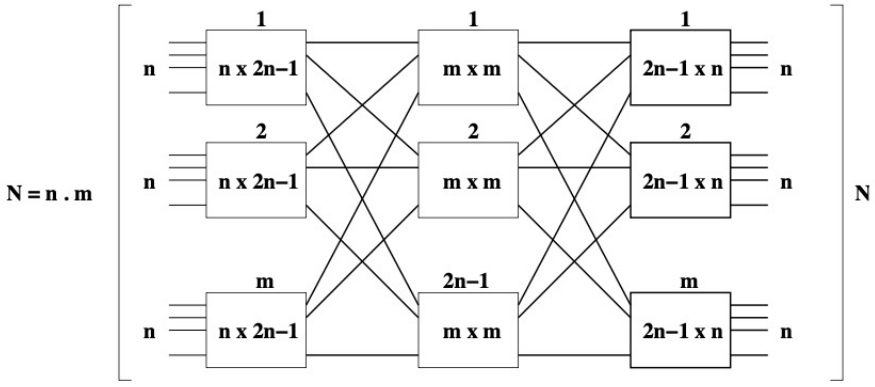


Figure 5.2: Detailed 3-Stage Clos Network Structure

For optimal 3-stage Clos networks, parameters m and n are typically chosen to minimize the total number of crosspoints while ensuring the non-blocking property. The application calculates these

parameters based on the total network size N. If n and m are not provided by the user, the application derives them. Based on the logic in 'baseClosCalculation' in 'Calculator.swift', the calculation for m involves rounding the square root of $N/2$, and n is derived by taking the ceiling of N/m, with an adjustment to ensure that the product $m \times n$ is at least N. This ensures that the network can accommodate N inputs. The relationships are approximately:

$$m \approx \sqrt{N/2}$$

$$n \approx \lceil N/m \rceil$$

(with adjustment $n \leftarrow n + 1$ while $mn < N$).

The total number of crosspoints in a 3-stage Clos network is the sum of the crosspoints in each stage. There are m input switches, each of size $n \times (2n - 1)$; $(2n - 1)$ middle switches, each of size $m \times m$; and m output switches, each of size $(2n - 1) \times n$. The total number of crosspoints $(C(3))$ is given by:

$$C(3) = m \cdot n \cdot (2n - 1) + (2n - 1) \cdot m^2 + m \cdot (2n - 1) \cdot n$$

This simplifies to:

$$C(3) = 2mn(2n - 1) + m^2(2n - 1)$$

This formula is used in the 'baseClosCalculation' function and as the base case in the recursive 'computeClosTopology' function in 'Calculator.swift'.

For k-stage Clos networks where $k > 3$ and k is odd, the middle stage is recursively treated as a $(k-2)$-stage Clos network. The total number of crosspoints in a k-stage Clos network is calculated recursively. In the application's 'computeClosTopology' function, for $k > 3$, the total crosspoints are calculated as the sum of crosspoints in the input and output stages plus the total crosspoints of the nested $(k - 2)$-stage network in the middle stage. The nested network effectively handles m inputs and outputs.

$$C(k) = 2mn(2n - 1) + (2n - 1) \cdot C_{(k-2)\text{-stage with input } m}$$

where m and n are the parameters for the k-stage network. This recursive structure is implemented in the 'computeClosTopology' function in 'Calculator.swift'.

The number of switches in each stage and the dimensions of these switches are determined by m and n. For a 3-stage network:

- Stage 1: m switches of size $n \times (2n - 1)$
- Stage 2: $(2n - 1)$ switches of size $m \times m$
- Stage 3: m switches of size $(2n - 1) \times n$

For recursive networks, the inner stages' switch counts and dimensions are determined by the parameters of the nested Clos network.

A snippet of the Swift code implementing the Clos network calculations is shown in Figure 5.3 and 5.4.

```swift
// MARK: - baseClosCalculation
/// Optimal 3-stage Clos parameters for a given N:
///    • m = round(√(N/2)), then adjust so m·n ≥ N
///    • n = ceil(N/m)
///    • crosspoints = 2·m·n·(2n−1) + (2n−1)·m²
func baseClosCalculation(for N: Int) -> (m: Int, n: Int, crosspoints: Double) {
    let mEstimate = sqrt(Double(N) / 2.0)
    var m = max(1, Int(round(mEstimate)))
    var n = Int(ceil(Double(N) / Double(m)))
    while m * n < N {
        n += 1
    }
    let mid = 2 * n - 1
    let cp = 2.0 * Double(m) * Double(n) * Double(mid)
           + Double(mid) * Double(m * m)
    return (m, n, cp)
}
```

Figure 5.3: Screenshot of Swift Code for Clos Network Calculation (Snippet)

```swift
// MARK: — Clos

func computeClosTopology(
    NInput: Int?,
    nInput: Int?,
    mInput: Int?,
    stageCount k: Int
) -> ClosSwitchModel? {
    // Must have N, odd k ≥ 3
    guard let rawN = NInput, k >= 3, k % 2 == 1 else { return nil }

    // If both n,m missing => use baseClosCalculation
    let (m0, n0, cp0): (Int, Int, Double)
    if nInput == nil && mInput == nil {
        (m0, n0, cp0) = baseClosCalculation(for: rawN)
    } else {
        // Otherwise derive missing of n/m if needed
        var N = rawN, n = nInput, m = mInput
        let provided = [n, m].compactMap{ $0 }.count
        if provided == 1 {
            if let nval = n {
                let mCalc = Int(ceil(Double(N) / Double(nval)))
                m = max(1, mCalc)
                N = nval * m!
            } else if let mval = m {
                let nCalc = Int(ceil(Double(N) / Double(mval)))
                n = max(1, nCalc)
                N = n! * mval
            }
        }
        guard let nn = n, let mm = m, N == nn * mm else { return nil }
        let mid = 2 * nn - 1
        let cp = 2.0 * Double(mm) * Double(nn) * Double(mid)
               + Double(mid) * Double(mm * mm)
        (m0, n0, cp0) = (mm, nn, cp)
    }

    // Now build the model
    let midCount = 2 * n0 - 1
    var totalCP = cp0
    var nested: ClosSwitchModel? = nil

    if k > 3 {
        // recurse on the "middle" size = m0
        nested = computeClosTopology(
            NInput: m0,
            nInput: nil,
            mInput: nil,
            stageCount: k - 2
        )
        totalCP = 2.0 * Double(m0) * Double(n0) * Double(midCount)
                + Double(midCount) * (nested?.crosspoints ?? 0)
    }

    return ClosSwitchModel(
        networkSize: rawN,
        stages: k,
        inputPorts: m0,
        outputPorts: m0,
        middleStageSwitches: midCount,
        crosspoints: totalCP,
        nestedTopology: nested
    )
}
```

Figure 5.4: Screenshot of Swift Code for Clos Network Calculation (Snippet)

5.2.2 Slepian Network Design

The Slepian network, also known as the Beneš network, is a type of rearrangeable non-blocking network [3, 4]. Similar to the Clos network, it is a multi-stage network, and its structure can also be defined recursively. A common representation of a Slepian network also involves multiple stages, and for recursive structures ($k > 3$), the middle portion contains a smaller Slepian network.

A key characteristic of the Slepian network is the relationship between the total network size N, the inputs per switch in the first stage (n), and the number of switches in the first stage (r). These parameters are related by $N = n \cdot r$. A general representation of a Slepian network topology

is shown in Figure 5.5.

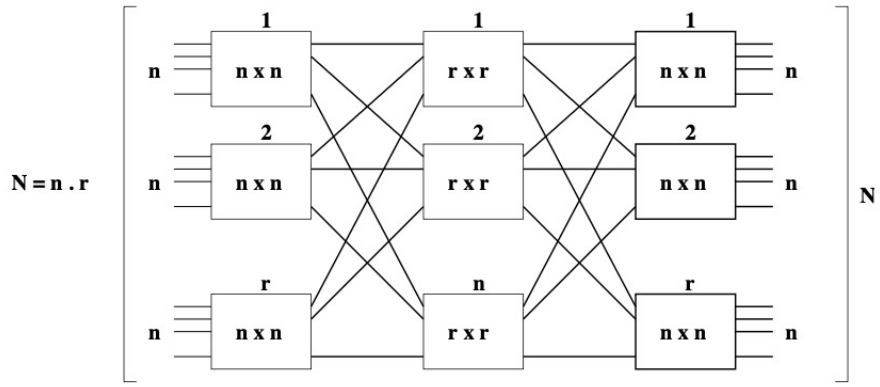


Figure 5.5: General Slepian Network Topology

The application's 'computeSlepianTopology' function handles the calculation of Slepian network parameters. If n and r are not provided, they are derived from N aiming for $N = n \cdot r$, similar to the Clos network, and adjusting N if necessary.

The calculation for the total number of crosspoints in a Slepian network, based on the implementation in 'Calculator.swift', utilizes different approaches depending on the stage count and input method. When n and r are provided or derived for the initial stage (and $k = 3$), an initial crosspoint value is considered, which in the code comment resembles $N(2n + r)$. However, for recursive Slepian networks with $k > 3$ stages, the total number of crosspoints is computed using a power formula relating the number of stages k, the inputs per switch n, and the number of input switches r:

$$C(k) = r^2 \cdot n^k$$

This formula is used for $k > 3$ in the 'computeSlepianTopology' function in 'Calculator.swift'. The middle stage of a k-stage Slepian network ($k > 3$) is treated as a $(k - 2)$-stage Slepian network, with recursive calls contributing to the total crosspoints, effectively following this power relationship.

The number of switches per stage and the dimensions of the switches in a Slepian network differ from Clos networks. In a Slepian network with parameters n and r:

- The number of switches in each stage is n.
- The dimensions of the switches vary by stage in the unfolded view. For a 3-stage network, the input and output stage switches are $n \times r$, and the middle stage switches are $r \times r$. However, when viewed recursively, the middle stage of a k-stage network ($k > 3$) consists of r switches, each representing a $(k - 2)$-stage Slepian network that handles n inputs and outputs.

A snippet of the Swift code implementing the Slepian network calculations is shown in Figure 5.6.

```swift
func computeSlepianTopology(
    NInput: Int?,
    nInput: Int?,
    rInput: Int?,
    stageCount k: Int
) -> SlepianSwitchModel? {
    guard let rawN = NInput, k >= 3 else { return nil }

    let (r0, n0, cp0): (Int, Int, Double)
    if nInput == nil && rInput == nil {
        // base Slepian: n≈√(N/2), r=ceil(N/n)
        let nEst = Int(round(sqrt(Double(rawN)/2.0)))
        var nCalc = max(1, nEst)
        var rCalc = Int(ceil(Double(rawN) / Double(nCalc)))
        while nCalc * rCalc < rawN { rCalc += 1 }
        let cpBase = 2 * sqrt(2) * pow(Double(rawN), 1.5)
        (r0, n0, cp0) = (rCalc, nCalc, cpBase)
    } else {
        // derive missing n/r if only one provided
        var N = rawN, n = nInput, r = rInput
        let provided = [n, r].compactMap{ $0 }.count
        if provided == 1 {
            if let nval = n {
                let rCalc = Int(ceil(Double(N)/Double(nval)))
                r = max(1, rCalc)
                N = nval * r!
            } else if let rval = r {
                let nCalc = Int(ceil(Double(N)/Double(rval)))
                n = max(1, nCalc)
                N = n! * rval
            }
        }
        guard let nn = n, let rr = r, N == nn * rr else { return nil }
        let cp = Double(rawN)*(2.0*Double(nn)+Double(rr))
        (r0, n0, cp0) = (rr, nn, cp)
    }

    var totalCP = cp0
    var nested: SlepianSwitchModel? = nil

    if k > 3 {
        nested = computeSlepianTopology(
            NInput: rawN,
            nInput: nil,
            rInput: nil,
            stageCount: k - 2
        )
        totalCP = pow(Double(r0), 2) * pow(Double(n0), Double(k))
    }

    return SlepianSwitchModel(
        networkSize: rawN,
        stages: k,
        inputPorts: r0,
        outputPorts: r0,
        middleStageSwitches: n0,
        crosspoints: totalCP,
        nestedTopology: nested
    )
}
```

Figure 5.6: Screenshot of Swift Code for Slepian Network Calculation (Snippet)

5.2.3 Parameter Derivation and User Input Handling

The application's `InputView.swift` includes logic to derive a missing parameter (N, n, or m/r) if the user provides only two of them. This ensures that the provided or derived parameters are consistent with the relationship $N \approx n \cdot m$ for Clos networks and $N = n \cdot r$ for Slepian networks, adjusting N upwards if necessary to meet this condition. This allows for flexibility in user input. The number of stages is directly selected by the user using a slider in the UI.

5.2.4 Optimization Logic

The primary optimization performed by the algorithms, as implemented, is the derivation of n and m (or r) values for a given N that are suitable for constructing the network, particularly when the user does not specify these values. While the concept of optimizing the *number of stages* for a given N to minimize crosspoints is a related topic in network design, the application's current implementation calculates the parameters and crosspoints for the number of stages explicitly selected by the user.

5.2.5 Predefined Parameters

- Maximum supported network size: The application is designed to handle network sizes up to a certain limit, influenced by computational performance and visualization constraints.
- Recursive configurations up to 9 stages for Clos switches and similar for Slepian switches are supported through the recursive nature of the calculation functions, allowing for visualization of complex multi-stage networks as defined by the user's stage selection.

5.2.6 Development Methodology

The project followed a phased development methodology:

- Phase 1: Initial Research and Planning
- Phase 2: UI/UX Design and Prototyping
- Phase 3: Core App Implementation (including algorithm development and initial visualization integration)
- Phase 4: Completion of Core Features and Beta Version (refining algorithms, visualization, and error handling)
- Phase 5: A/B Testing and Feature Refinements (gathering user feedback and implementing improvements)
- Phase 6: Final Testing and Documentation

This iterative approach allowed for continuous testing and refinement of the application's features and performance.

5.2.7 Model Parameters

The data structures within the application (`SwitchModels.swift`) store the calculated parameters for each network topology, including:

Clos Switch Model:

- Input: Total network size (N), Number of stages (k).
- Output: Inputs per switch (n), Number of input switches (m), Number of middle stage switches ($2n - 1$), Total crosspoint count, and a nested topology model if $k > 3$.

Slepian Switch Model:

- Input: Total network size (N), Number of stages (k).
- Output: Inputs per switch (n), Number of input switches (r), Number of middle stage switches (n), Total crosspoint count, and a nested topology model if recursive.

Visualization Parameters:

- The visual dimensions and layout of the topology are dynamically adjusted based on the calculated number of switches in each stage.
- Color-coded connections are used to represent the interconnections between switches and stages, enhancing clarity in the visualization (implemented in ClosSwitcView.Swift and SlepianSwitchView.swift).

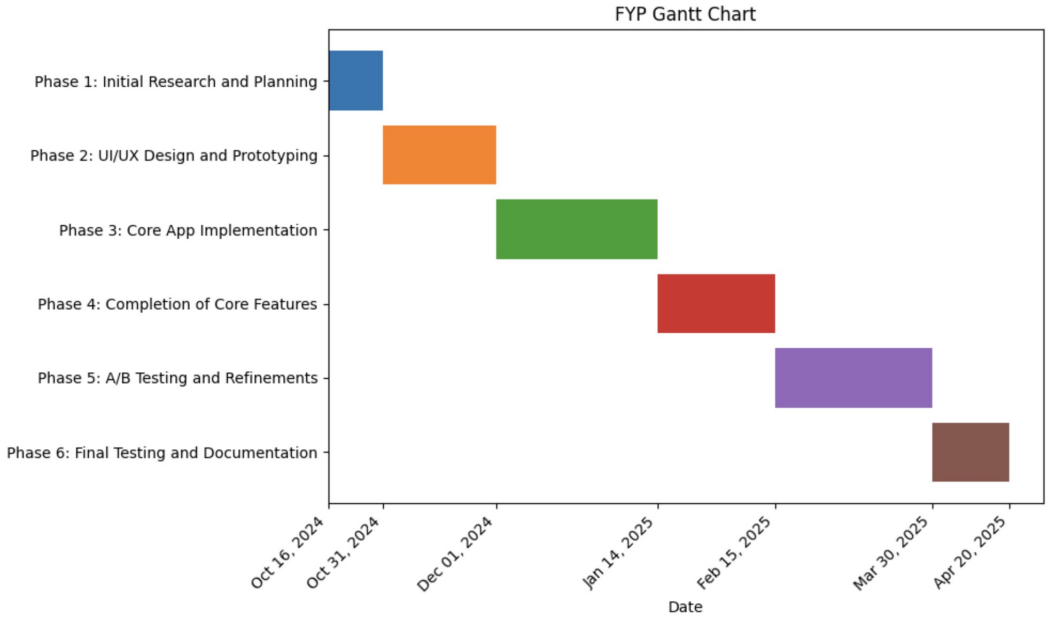
5.2.8 Validation and Testing

The application's outputs were validated against theoretical calculations and examples from the lecture materials to ensure the accuracy of the implemented algorithms. Performance testing focused on evaluating the computational efficiency of the algorithms for varying network sizes and ensuring smooth rendering of the visualizations. Usability testing was conducted to refine the user interface and overall user experience.

Chapter 6: Project Workplan

6.1 Project Timeline and Phases

This project is planned to span six months, divided into six distinct phases, ensuring a systematic approach to development and delivery.



6.1.1 Phase 1: Initial Research and Planning (16 October – 31 October)

Goal: Gain a deep understanding of network topology technologies and define the app structure.

- Research Clos and Slepian switches and related network topologies.
- Identify features to include in the app, such as:
  - Topology creation
  - Visualization
  - Optimization
- Explore Swift frameworks and decide between SwiftUI and UIKit.
- Create a detailed project plan and specification document.
- Consult the supervisor for feedback and final approval.

6.1.2 Phase 2: UI/UX Design and Prototyping (1 November – 30 November)

Goal: Create the user interface layout and app flow.

- Design wireframes and low-fidelity prototypes using Figma or SwiftUI.
- Develop basic app views:
  - Network input screen
  - Topology display screen (interactive visualization)
  - Settings and help screens
- Present the prototype to the supervisor and collect feedback.

6.1.3 Phase 3: Core App Implementation (1 December - 14 January)

Goal: Implement the core functionality of the app, focusing on backend logic and visualization, give a brief pause for example period.

- Develop logic for processing user inputs and network parameters.
- Implement algorithms for Clos and Slepian switch topologies.
- Integrate SwiftUI's Canvas API or Core Graphics for visualizing network designs.
- Write modular code for switch size and stage number calculations.
- Conduct regular testing to ensure feature stability.

6.1.4 Phase 4: Completion of Core Features and Beta Version (15 January – 15 February)

Goal: Finalize the app's functionality and prepare for beta testing.

- Complete backend logic and topology calculations.
- Enhance visualization features for handling larger networks.
- Implement error handling and optimize app performance.
- Refine the UI based on pre-exam feedback.
- Release a beta version for testing by peers and supervisor.

6.1.5 Phase 5: A/B Testing and Feature Refinements (15 February – 30 March)

Goal: Gather feedback from A/B testing and refine the app.

- Conduct A/B testing on key features (e.g., visual style, ease of use).
- Collect feedback on usability, functionality, and improvement areas.
- Refine the app based on feedback:
  - Optimize performance.
  - Improve UI elements.
  - Add advanced features like topology export options.

6.1.6 Phase 6: Final Testing and Documentation (1 April – 20 April)

Goal: Complete testing, debugging, and project documentation.

- Perform final testing across different iOS devices and versions.
- Document the app comprehensively, including user instructions.
- Finalise the project and prepare for presentations and reports.

6.2 Summary of the Workplan

The outlined project phases provides a structured roadmap to how I achieved the goals of the iOS app. Each phase builds upon the previous one, ensuring steady progress and integration of user feedback. This plan ensures a balanced approach, accommodating academic responsibilities while delivering a robust and functional application.

Chapter 7: Summary and Conclusions

This report details the development of an iOS application designed for the visualization and optimization of Clos and Slepian switching network topologies. The project aimed to create an accessible tool for network engineers, researchers, and students, bridging the gap between theoretical network design and practical understanding through interactive visualization and automated parameter calculation.

Throughout the project lifecycle, significant progress was made towards achieving the objectives outlined in the project specification (Chapter: 1). All the defined Core Goals have been successfully completed:

- **User Input Handling**: The application effectively captures user inputs for network size, stage count, and switch type, with built-in logic to derive missing parameters ($N, n, m/r$) when necessary, ensuring valid input combinations[1].
- **Algorithm Implementation**: Robust algorithms for calculating the parameters and total crosspoints for both Clos and Slepian networks, including their recursive multi-stage configurations, have been implemented in Swift (`Calculator.swift`)[1].
- **Visualization**: An interactive visualization engine utilizing SwiftUI's Canvas API has been successfully integrated, dynamically rendering the calculated network topologies and allowing users to explore the structure and connections[1].
- **Error Handling**: Measures have been implemented to handle invalid user inputs and computational edge cases, ensuring the application's stability and preventing crashes[1].
- **Modular Code**: The codebase is structured with a modular approach, separating concerns such as calculation logic (`Calculator.swift`), data models (`SwitchModels.swift`), and user interface components, which facilitates maintainability and potential future expansion[1].

7.1 Future Work

While the core objectives of the project have been met, there remain several avenues for future development to enhance the application's functionality and performance. Addressing some of the Advanced Goals outlined in the project specification and incorporating additional features would further increase the tool's value[1].

7.1.1 Remaining Objectives and Advanced Features

- **Advanced Features (Export/Share)**: Implementing functionality to export network topologies or share network parameters (e.g., as an image, PDF, or structured data format) would be a valuable addition for documentation and collaboration purposes. A share sheet integration in iOS would provide a native way to share the generated topology[1].
- **Performance Optimization**: While the current algorithms perform adequately for the supported network sizes, further performance profiling and optimization could be explored, especially for handling extremely large networks or rendering highly complex recursive structures[1]. Investigating alternative calculation methods or visualization techniques might yield further improvements.
- **Automated Stage Optimization**: The current application requires the user to manually select the number of stages. A potential enhancement would be to implement an algorithm that automatically determines the optimal number of stages for a given network size to minimize crosspoints, building on theoretical principles discussed in the literature.

7.2 Testing and Feedback

Comprehensive testing was conducted throughout the development process to ensure the application's accuracy, performance, and usability. This included unit testing of the core calculation algorithms and manual testing of the user interface and workflow.

7.2.1 User Feedback

To gather valuable insights into the application's usability, clarity, and perceived performance, user feedback was actively sought. This included controlled testing where users interacted with the application, providing commentary on their experience. For example, as a part of this, certain participants were provided with a text-based tutorial on using the app, while others received a video-based tutorial. The feedback indicated that the group who received the video-based tutorial generally reported a better understanding and a more positive initial experience with the application. This process was instrumental in identifying areas for improvement in the user interface, the clarity of the provided information about the network topologies, and the overall ease of use for individuals with varying levels of familiarity with switching concepts. The feedback highlighted specific elements of the visualization and input process that could be refined to enhance the user experience.

7.2.2 Industry Feedback

Additionally, invaluable feedback was received from a friend and colleague at PwC. Gaining insights from someone working in the technology industry provided a practical perspective on the application's relevance, potential use cases, and areas for improvement from a professional standpoint. This industry feedback helped validate the project's foundational concepts and offered suggestions for features or optimizations that would be most impactful in real-world scenarios. The opportunity to discuss the project with an industry expert was highly beneficial in shaping the final direction and understanding the practical implications of the developed tool.

7.2.3 Conclusion

In conclusion, this project successfully delivered an iOS application for designing and visualizing Clos and Slepian switching networks, meeting all of its core objectives[cite: 238]. The application provides a functional and intuitive platform for users to explore these fundamental network topologies, combining algorithmic computation with interactive visualization. The development process, guided by a structured work plan and informed by user feedback and expert insights, resulted in a robust and user-friendly tool. This is particularly relevant for the design of large-scale networks commonly found in modern data centers and telecommunications infrastructure, where Clos and Slepian switches play a critical role due to their scalability and non-blocking or rearrangeable properties. By automating the complex calculations and providing a clear, interactive visual representation on a portable device, the application significantly enhances the efficiency and user-friendliness of modeling these large and intricate network structures compared to manual methods or less specialized tools. While opportunities for future work exist to further enhance the application's capabilities and performance, the current implementation serves as a solid foundation and a valuable tool for understanding and working with Clos and Slepian networks. The project provided a significant learning experience, from delving into complex networking theory to applying modern mobile application development frameworks.

Acknowledgements

This project would not have been possible without the support and guidance of several individuals and institutions. I would like to express my sincere gratitude to everyone who contributed to the successful completion of this work.

Foremost, I extend my deepest thanks to my supervisor, **Professor John Murphy**. His expert guidance, insightful feedback, and unwavering support throughout the entire project were invaluable. Professor Murphy's profound understanding of networking concepts, particularly the intricacies of Clos and Slepian switches, was instrumental in shaping the direction of this research and development. His patient explanations helped demystify complex theoretical aspects and provided clarity on the practical implications of the project. His encouragement and availability for discussion were crucial at every stage, from initial conceptualization to final implementation and reporting.

I am also incredibly grateful to **Nikhil Keswani** at PwC for serving as my industry mentor. His real-world perspective and practical insights into the technology industry provided a vital link between academic theory and industry practice. His feedback on the application's relevance and potential utility in a professional environment was highly valuable and helped refine the project's focus. I appreciate him taking the time to share his expertise. Email ID: keswani06@gamil.com

A special thank you to all the **beta testers** who dedicated their time and effort to test the application. Their participation in the controlled testing, providing honest and constructive feedback, was essential for identifying usability issues, validating functionality, and improving the overall user experience. The insights gathered from their interactions were directly incorporated into the final version of the application, making it more robust and intuitive.

Finally, I would like to thank **University College Dublin (UCD)** for providing the opportunity to undertake this final year project. The resources, academic environment, and foundational knowledge gained during my time at UCD were fundamental to my ability to complete this work.

Bibliography

1. Clos, C. A Study of Non-Blocking Switching Networks. *The Bell System Technical Journal* **32,** 406–424 (1953).
2. Prof. Murphy, J. *Switching Concepts Lecture Notes* Used as base knowledge on the switches. 2024. https://drive.google.com/file/d/1-mD_artFfNSXPL4wEH5kyzWNkwm-dfZN/view?usp=share_link.
3. Beneš, V. E. *Mathematical Theory of Connecting Networks and Telephone Traffic* (Academic Press, 1962).
4. Slepian, D. *Two theorems on a particular crossbar switching network* Unpublished manuscript. 1952.
5. https://developer.apple.com/documentation/swiftui/.
6. https://developer.apple.com/documentation/swiftui/canvas.
7. *Network Topology: Definition, Types, Advantages and Disadvantages - Netmonk* https://netmonk.id/blog/network-topology-definition-types-advantages-and-disadvantages.
8. *An Overview: Basic Concept of Network Simulation Tools - ijarcce* https://ijarcce.com/upload/2017/si/ICITCSA-17/IJARCCE-ICITCSA%205.pdf.
9. *Network Diagram Generators: Free vs. Paid Solutions - NetSecCloud* https://netseccloud.com/network-diagram-generators-free-vs-paid-solutions.